

Market-Basket Analysis on the IMDb Movies Dataset

Algorithms for Massive Data – Project 2

Zeinab Abbas

Master in Data Science for Economics

Abstract

This project implements a market-basket analysis system on the IMDb Movies Dataset. Each movie is treated as a basket, while the actors listed in the `Star1`, `Star2`, `Star3`, and `Star4` fields are treated as items. The goal is to identify frequent actor combinations and derive association rules from them. The Apriori algorithm is implemented to discover frequent 1-itemsets, 2-itemsets, and 3-itemsets. The PCY algorithm is also implemented as an optimization for frequent pair mining. The experimental results show that PCY reduces the number of candidate pairs from 139 to 28 while preserving the same frequent pairs found by Apriori. Although PCY is slower on this relatively small dataset because of hashing overhead, the candidate reduction demonstrates its scalability advantage for larger datasets.

1 Introduction

Market-basket analysis is a data mining technique used to discover frequent co-occurrence patterns in transactional data. In the classical setting, each transaction is a basket and each product inside the basket is an item. The objective is to identify groups of items that appear together frequently and, optionally, derive association rules describing conditional relationships between them.

In this project, the same idea is applied to the IMDb Movies Dataset. Each movie is considered a basket, and the actors listed in the four main actor columns are treated as items. Therefore, a frequent itemset corresponds to a group of actors that repeatedly appear together in movies.

The project focuses on two techniques studied in the course:

- the Apriori algorithm, used to find frequent itemsets;
- the PCY algorithm, used to reduce the number of candidate pairs through hashing and bitmap filtering.

The implementation is written in Python 3 using a Jupyter notebook. The dataset is downloaded using the Kaggle API, making the experiments reproducible.

2 Dataset

2.1 Dataset Description

The dataset used in this project is the IMDb Top 1000 Movies Dataset available on Kaggle. The dataset contains 1000 movies and 16 attributes. The relevant fields for this project are:

- `Series_Title`: title of the movie;

- **Star1**: first main actor;
- **Star2**: second main actor;
- **Star3**: third main actor;
- **Star4**: fourth main actor.

2.2 Data Organization

The data is transformed into a transactional representation. Each movie becomes one basket containing the four actors appearing in the columns **Star1**, **Star2**, **Star3**, and **Star4**. For example, a movie row is converted into a basket of the form:

$$\{\text{Actor}_1, \text{Actor}_2, \text{Actor}_3, \text{Actor}_4\}.$$

The complete dataset contains:

- 1000 baskets;
- 2709 unique actors.

3 Preprocessing

The preprocessing phase consists of selecting the actor columns and converting each movie into a list of actors. Since the market-basket model only requires baskets and items, all other columns are ignored for the mining phase.

The following transformation is applied:

$$\text{Movie} \rightarrow \{\text{Star1}, \text{Star2}, \text{Star3}, \text{Star4}\}.$$

The support of each actor is then counted. The support of an itemset is the number of baskets in which that itemset appears. In this project, the minimum support threshold is set to:

$$\text{MIN_SUPPORT} = 5.$$

This means that an actor or actor combination must appear in at least five movies to be considered frequent. This threshold removes very rare actor combinations while still keeping enough frequent itemsets to demonstrate the algorithms.

Figure 1 shows the distribution of actor supports.

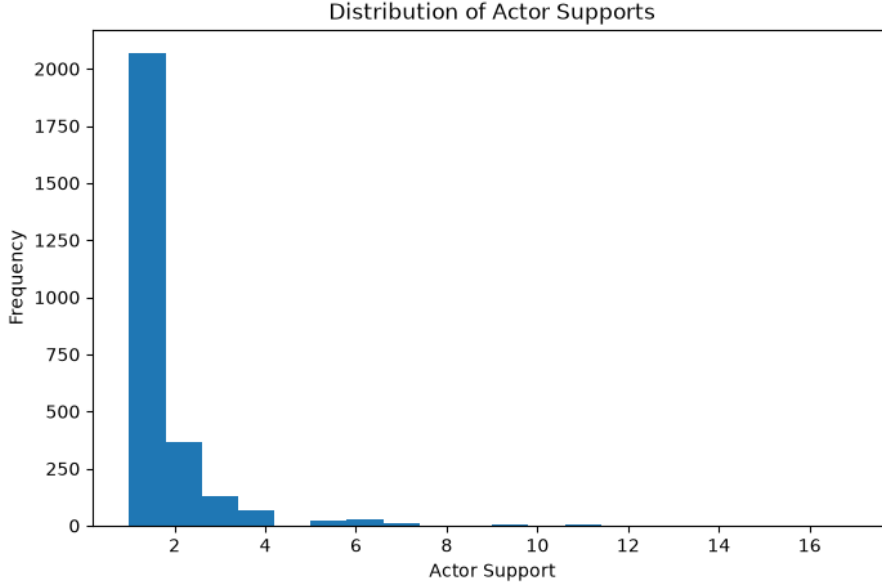


Figure 1: Distribution of actor supports. Most actors appear only once or twice.

The support distribution is highly skewed. Most actors appear in only a few movies, while only a small number appear frequently enough to pass the minimum support threshold.

4 Apriori Algorithm

4.1 Methodology

The Apriori algorithm is used to discover frequent itemsets. Its main principle is that if an itemset is frequent, then all of its subsets must also be frequent. Conversely, if an itemset is not frequent, then no larger itemset containing it can be frequent.

The implementation follows these steps:

1. count the support of all individual actors;
2. keep only frequent 1-itemsets;
3. generate candidate 2-itemsets from frequent actors;
4. count their supports;
5. keep only frequent 2-itemsets;
6. extend the process to candidate 3-itemsets using Apriori pruning.

4.2 Frequent 1-Itemsets

The first pass counts how often each actor appears in the dataset. Out of 2709 unique actors, only 79 satisfy the minimum support threshold of 5. These actors form the frequent 1-itemsets, denoted as L_1 .

$$|L_1| = 79.$$

This confirms that most actors are rare and are removed early by the Apriori pruning process.

4.3 Frequent 2-Itemsets

Using the frequent actors in L_1 , candidate actor pairs are generated. Apriori generated 139 candidate pairs. After applying the minimum support threshold, only 3 frequent 2-itemsets remained.

Frequent Pair	Support
Daniel Radcliffe and Rupert Grint	6
Daniel Radcliffe and Emma Watson	5
Emma Watson and Rupert Grint	5

Table 1: Frequent 2-itemsets discovered by Apriori.

These pairs correspond to actors who repeatedly appear together in the Harry Potter movies.

4.4 Frequent 3-Itemsets

Apriori was also extended to discover frequent 3-itemsets. Candidate triples are generated only if all their 2-item subsets are already frequent. This follows the Apriori pruning principle.

Only one candidate triple was generated, and it was frequent:

Frequent Triple	Support
Daniel Radcliffe, Rupert Grint, Emma Watson	5

Table 2: Frequent 3-itemset discovered by Apriori.

This result is consistent with the frequent pairs previously discovered.

5 PCY Algorithm

5.1 Methodology

The PCY algorithm is an optimization of Apriori for frequent pair mining. It reduces the number of candidate pairs by using hashing during the first pass over the dataset.

During the first pass, each pair of actors is hashed into a bucket. Instead of storing all candidate pairs immediately, the algorithm counts how many pairs fall into each bucket. After the first pass, a bitmap is created:

$$\text{bitmap}[i] = \begin{cases} 1 & \text{if bucket } i \text{ has support at least } s, \\ 0 & \text{otherwise.} \end{cases}$$

During the second pass, a pair is considered as a candidate only if:

1. both actors are frequent individual items;
2. the bucket into which the pair hashes is frequent according to the bitmap.

This reduces the number of candidate pairs that need to be counted explicitly.

5.2 PCY Results

Using 3000 hash buckets, PCY reduced the number of candidate pairs from 139 to 28. Importantly, PCY still found the same 3 frequent pairs as Apriori.

Algorithm	Candidate Pairs	Frequent Pairs
Apriori	139	3
PCY	28	3

Table 3: Comparison between Apriori and PCY.

This shows that PCY successfully reduces the search space without changing the final frequent itemsets.

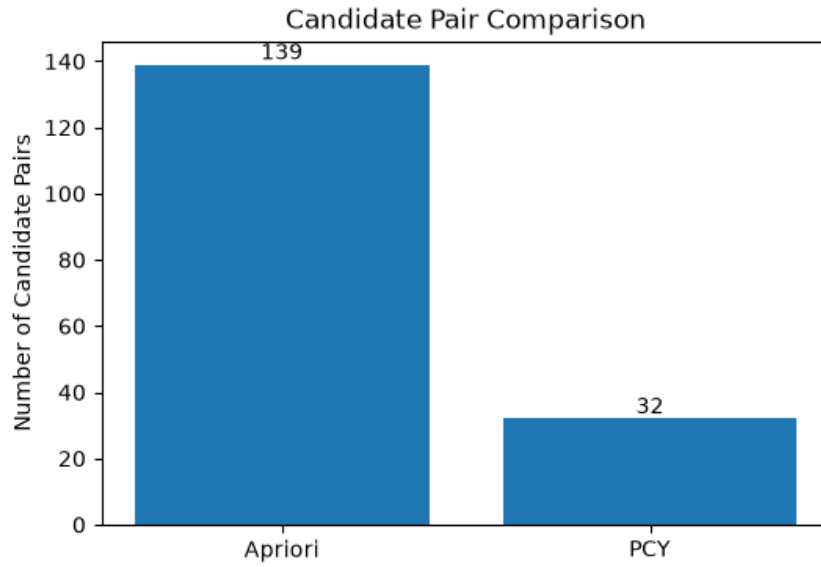


Figure 2: Comparison of candidate pairs generated by Apriori and PCY.

6 Association Rules

After finding frequent pairs, association rules are generated. For a rule $A \rightarrow B$, the confidence is defined as:

$$\text{confidence}(A \rightarrow B) = \frac{\text{support}(A \cup B)}{\text{support}(A)}.$$

Confidence measures how often actor B appears when actor A appears.

Lift is also computed:

$$\text{lift}(A \rightarrow B) = \frac{\text{confidence}(A \rightarrow B)}{\text{support}(B)/N},$$

where N is the total number of baskets. Lift greater than 1 indicates that the co-occurrence is stronger than expected under independence.

Rule	Support	Confidence	Lift
Rupert Grint $\rightarrow$ Daniel Radcliffe	6	1.000	166.67
Daniel Radcliffe $\rightarrow$ Rupert Grint	6	1.000	166.67
Rupert Grint $\rightarrow$ Emma Watson	5	0.833	119.05
Daniel Radcliffe $\rightarrow$ Emma Watson	5	0.833	119.05
Emma Watson $\rightarrow$ Daniel Radcliffe	5	0.714	119.05
Emma Watson $\rightarrow$ Rupert Grint	5	0.714	119.05

Table 4: Association rules generated from frequent actor pairs.

The strongest rules have confidence equal to 1.0, meaning that whenever Rupert Grint appears in a movie, Daniel Radcliffe also appears, and vice versa. The lift values are much greater than 1, indicating strong non-random co-occurrence.

7 Experimental Evaluation

7.1 Runtime Comparison

The runtime of Apriori and PCY was measured using Python’s `time.perf_counter()` function. Both algorithms were executed on the same dataset and with the same minimum support threshold.

Algorithm	Runtime (seconds)
Apriori	0.0015
PCY	0.0046

Table 5: Runtime comparison between Apriori and PCY.

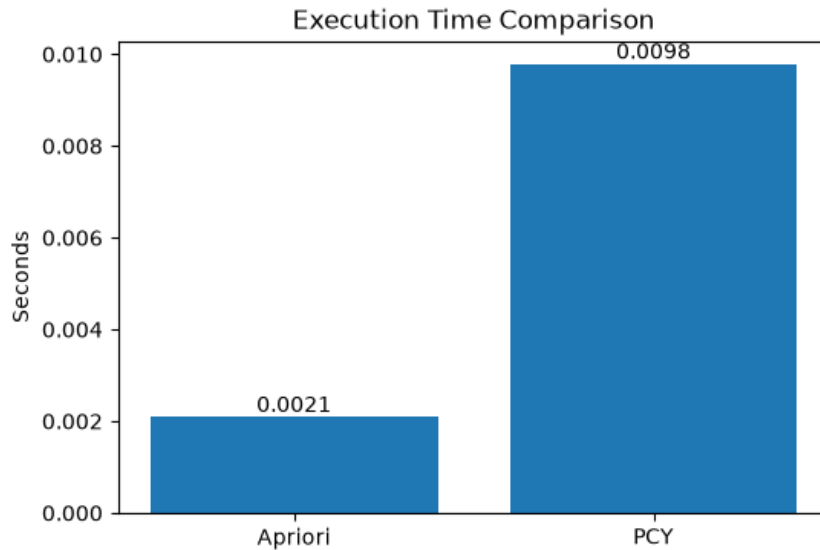


Figure 3: Execution time comparison between Apriori and PCY.

Apriori is faster on this small dataset. This is expected because PCY introduces additional overhead through hashing and bitmap construction. Since the dataset contains only 1000 baskets, the overhead is greater than the computational savings.

7.2 Scalability Experiment

To evaluate scalability, both algorithms were executed on progressively larger subsets of the dataset:

200, 400, 600, 800, 1000 movies.

For each subset, the execution time and the number of generated candidate pairs were recorded.

Dataset Size	Apriori Time	PCY Time	Apriori Candidates	PCY Candidates
200	0.000124	0.000943	0	0
400	0.000248	0.001821	6	0
600	0.000336	0.002160	19	1
800	0.000484	0.002857	54	7
1000	0.000625	0.004198	139	28

Table 6: Scalability experiment results.

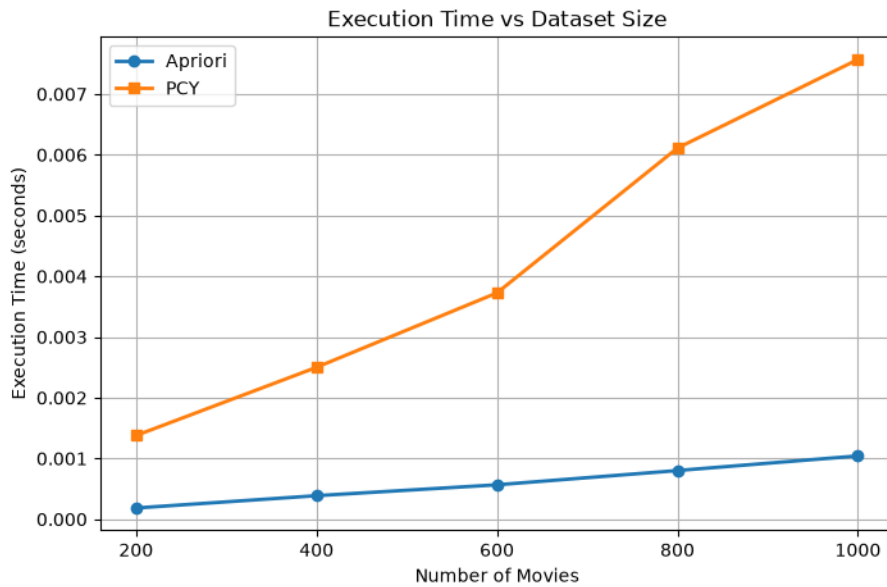


Figure 4: Execution time as the dataset size increases.

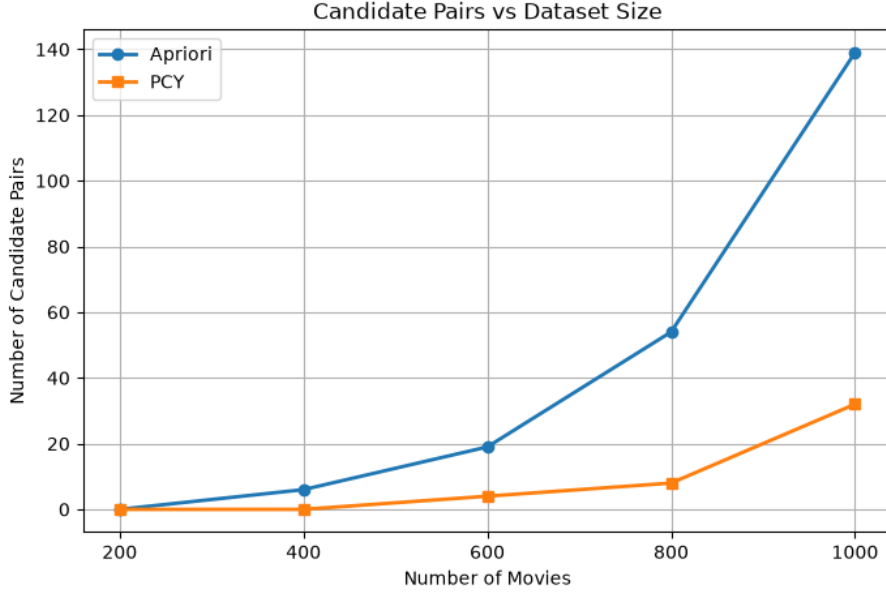


Figure 5: Number of candidate pairs as the dataset size increases.

The scalability experiment shows that both algorithms require more time as the dataset size increases. However, the most important difference is observed in candidate generation. Apriori generates increasingly many candidate pairs as the dataset grows, while PCY filters many of them during the hashing phase.

For the complete dataset of 1000 movies, Apriori generated 139 candidate pairs, while PCY generated only 28. This corresponds to an approximate 80% reduction in the number of candidate pairs.

Although PCY remains slower on this relatively small dataset, the reduction in candidate generation demonstrates its advantage for larger-scale datasets, where candidate generation and memory usage become more important.

8 Replicability

The project is implemented as a Jupyter notebook executable in Python 3. The dataset is downloaded from Kaggle using the Kaggle API. To avoid exposing sensitive credentials, the final notebook uses placeholder strings for the Kaggle username and key.

The main steps required to reproduce the experiments are:

1. install the required Python packages;
2. configure Kaggle API credentials;
3. run the notebook from top to bottom;
4. execute the dataset download cell;
5. run the preprocessing, algorithm, and experiment cells.

The experiments are deterministic except for small variations in runtime caused by the execution environment.

9 Discussion

The results confirm that market-basket analysis can identify meaningful co-occurrence patterns in movie cast data. The frequent itemsets mainly correspond to actors appearing together in the Harry Potter film series. This is a reasonable and interpretable result, showing that the implemented algorithms detect real recurring collaborations.

Apriori works well on this dataset because the number of baskets is relatively small. However, the candidate pair comparison shows that Apriori still generates more candidates than PCY. PCY reduces this number significantly by eliminating pairs that hash into infrequent buckets.

The runtime comparison shows that PCY is slower than Apriori for this dataset. This does not contradict the theoretical motivation of PCY. PCY is designed for massive datasets, where reducing the number of candidate pairs can reduce memory usage and improve scalability. On a small dataset, the additional hashing overhead can dominate the execution time.

10 Conclusion

This project implemented a market-basket analysis system using the IMDb Movies Dataset. Each movie was treated as a basket, and the actors in the **Star1**, **Star2**, **Star3**, and **Star4** columns were treated as items.

The Apriori algorithm was implemented to discover frequent 1-itemsets, 2-itemsets, and 3-itemsets. Association rules were generated from the frequent pairs using confidence and lift. The PCY algorithm was also implemented and compared with Apriori.

The experimental results show that PCY reduced the number of candidate pairs from 139 to 28 while still finding the same frequent pairs as Apriori. Although PCY was slower on this relatively small dataset, the scalability experiment demonstrated its ability to reduce candidate generation as the dataset size increases.

Overall, the project satisfies the objective of implementing a frequent itemset mining system using techniques studied in the course and demonstrates both correctness and scalability considerations.

Declaration

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work, and including any code produced using generative AI systems. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion, or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

References

- [1] J. Leskovec, A. Rajaraman, and J. D. Ullman. *Mining of Massive Datasets*. Cambridge University Press.
- [2] Kaggle. *IMDb Movies Dataset*. Available through Kaggle datasets.